# Runbook de produção — go-live e operação do gc-prod

Documento técnico de vida longa: vale para o go-live (blocos 14.4 e 14.5 da Sprint 14) e para qualquer operação futura em gc-prod. Escrito em 2026-10-26 · Branch: `main`

---

> **Nota (2026-10-26):** gc-prod está fora do escopo por decisão do Breno, que não tem acesso a ele ("não vamos fazer nada em gc-prod, finalize tudo pelo dev"). Os blocos 14.4 e 14.5 são **ensaiados em gc-dev** (backup e restauração, importação, conciliação, fumaça); o que só existe em prod (domínio, Vercel de prod, backup do plano pago) continua só aqui. O restante do documento é registro para quem for operar prod.

## Leia antes

**Nada deste documento foi executado.** gc-prod (`zondscnfosubxutpshux`) está fora do escopo do repositório (ver `CLAUDE.md`), e a regra é executada: `scripts/comum/gc-dev-guard.mjs` aborta qualquer script que aponte para outro projeto. O que segue é **registro para quem for operar prod**, montado a partir do código, das migrations e dos documentos de entrega.

Estado conferido em 2026-10-26, pela API de gerenciamento da Supabase (só a listagem de projetos, sem conexão com o banco):

| PROJETO | REF | ESTADO | REGIÃO |
|---|---|---|---|
| gc-prod | `zondscnfosubxutpshux` | `INACTIVE` (pausado) | sa-east-1 |
| gc-dev | `gzbmhgnpoehormnidmgg` | `ACTIVE_HEALTHY` | sa-east-1 |

Os passos estão na ordem de execução. Cada passo diz como conferir que deu certo antes de seguir.

## 1. Pré-requisitos e autorizações

Nenhum passo começa sem os itens abaixo. Os da equipe são os da seção 3 de `docs/sprint-14/perguntas-para-a-equipe.md`.

| ITEM | DE QUEM | PARA QUÊ |
|---|---|---|
| Autorização escrita para trabalhar em gc-prod | Breno | muda a regra do `CLAUDE.md` para a janela do go-live |
| Acesso de dono à conta da Supabase que tem o gc-prod | equipe | reativar o projeto, ver o plano e os backups |

| ITEM | DE QUEM | PARA QUÊ |
|---|---|---|
| Janela de virada (sugestão: sexta à tarde) | equipe | migrations destrutivas e importação pedem o sistema parado |
| Planilha "Gestão Contratos.xlsx" final, congelada na data de corte | equipe | importação (14.2) e conciliação (14.3) |
| As cinco decisões do 14.1 | equipe | regras da importação (ver `14.2-status-entrega.md`, seção 6) |
| Lista dos usuários reais: nome, e-mail, telefone, perfil | equipe | convites (passo 7) |
| Domínio e acesso ao DNS | equipe | passo 11 |
| Plano da Supabase com backup automático | equipe | passo 2 e seção 12 |
| Sentry ou log da Vercel | equipe | passo 13 |

Ferramentas na máquina de quem opera: `npx supabase` (CLI), `pg_dump` / `psql` da mesma versão maior do Postgres do projeto, Node 20 e o repositório na `main` com a rodada do plano passando.

**Reativar o projeto.** Painel da Supabase → projeto gc-prod → *Restore project*. Conferir com:

```
curl -s -H "Authorization: Bearer $SUPABASE_ACCESS_TOKEN"
https://api.supabase.com/v1/projects
# gc-prod precisa aparecer como ACTIVE_HEALTHY
```

Pendente de confirmar: o gc-prod está pausado pelo menos desde 2026-09-05 (registro do 4.1). A Supabase limita a restauração de projeto pausado há muito tempo no plano gratuito. Se o painel não oferecer o *Restore*, o caminho é baixar o backup que o painel oferece e decidir com o Breno entre restaurar num projeto novo ou começar o gc-prod do zero. Isso não está decidido.

## 2. Backup antes de tudo

O backup é o rollback do go-live (seção 14). Sem backup conferido, nenhum passo seguinte começa.

**a. Backup do painel** (se o plano tiver): Database → Backups. Anotar a data e a hora do último backup diário. Se o plano tiver PITR, anotar o horário exato de início da janela.

**b. Dump lógico próprio**, independente do plano. A string de conexão vem do painel (Database → Connection string, modo *session*). Não gravar a senha em arquivo do repositório.

```
export PROD_DB_URL='postgresql://postgres.zondscnfosubxutpshux:<senha>@<host>:5432/postgres'
D=backup-gc-prod-$(date +%Y%m%d-%H%M)
mkdir -p ~/backups/$D && cd ~/backups/$D

pg_dump "$PROD_DB_URL" --schema-only  --no-owner -f schema.sql
pg_dump "$PROD_DB_URL" --data-only    --no-owner -n public -f dados-public.sql
pg_dump "$PROD_DB_URL" -Fc            --no-owner -f completo.dump
```

O Storage (anexos, NFs, comprovantes, documentos da automação) **não** entra no `pg_dump`: o banco só guarda o caminho do arquivo. Baixar os buckets à parte (painel ou API de Storage) e anotar a contagem de arquivos de cada um.

**c. Como verificar o backup.** Um backup só conta depois de lido de volta:

1. `pg_restore --list completo.dump | wc -l` devolve a lista de objetos, sem erro.
2. Contagem de linhas das tabelas principais em prod, anotada aqui antes de seguir:

```
select 'empresas' t, count(*) from empresas union all
select 'profiles', count(*) from profiles union all
select 'clientes', count(*) from clientes union all
select 'obras', count(*) from obras union all
select 'orcamentos', count(*) from orcamentos union all
select 'propostas', count(*) from propostas union all
select 'itens', count(*) from itens union all
select 'notas_fiscais', count(*) from notas_fiscais union all
select 'pagamentos', count(*) from pagamentos union all
select 'fd', count(*) from fd;
```

3. O teste de restauração da seção 12, num projeto descartável, é a prova de verdade. Para o go-live, basta o item 1 e o 2; o teste completo é atividade do 14.5.

## 3. Repair do histórico de migrations

O que o `supabase/README.md` registra: as migrations do repositório entraram nos ambientes antes de a CLI controlar o histórico. Em gc-dev, o `repair` das dez primeiras foi feito em 2026-09-05 (bloco 4.1). **Em gc-prod, nunca foi feito.** Um `db push` sem o `repair` tentaria reaplicar tudo, começando pela `revisao_schema`, que é destrutiva.

O estado do gc-prod é **desconhecido** e se descobre assim:

```
npx supabase link --project-ref zondscnfosubxutpshux
npx supabase migration list           # histórico remoto: espera-se vazio
```

E no SQL Editor de gc-prod, rodar `supabase/verificar_schema_aplicado.sql` (somente leitura). Ele diz quais das migrations 004 a 010 estão aplicadas, pelo catálogo do Postgres. Rodar o mesmo em gc-dev e comparar linha a linha.

- O `repair --status applied` entra **só** para as versões que o verificador mostrar como aplicadas (e a 001 a 003, se as tabelas existirem).
- O verificador **não cobre** da 011 em diante. Todas elas (36 arquivos) foram criadas a partir de 2026-09-05, quando o gc-prod já estava `INACTIVE` (registro do 4.1), então a hipótese é que nenhuma foi aplicada. Confirmar no catálogo antes do push: por exemplo, `propostas_historico`, `auditoria_eventos` e `pedidos_acesso` não devem existir.

```
# exemplo, se o verificador mostrar 001–010 aplicadas
npx supabase migration repair --status applied \
  20260424121550 20260424121551 20260424121552 \
  20260511151924 20260512135125 20260512141810 20260512151918 \
  20260901120909 20260901120910 20260901120911
npx supabase migration list            # as aplicadas aparecem nos dois lados
```

## 4. As migrations, em ordem

São **46** arquivos em `supabase/migrations/` (contados em 2026-10-26). O `db push` aplica na ordem do timestamp, que é a ordem abaixo. A ordem importa: a 004 recria `obras_com_valores` sem a coluna `progresso_itens_pct` que a 007 traz; invertidas, `/obras` quebra em runtime.

Legenda da coluna "Atenção":

- **DESTRUTIVA**: apaga coluna ou dado sem volta. Só backup desfaz.
- **dado**: altera ou preenche linhas que já existem (backfill).
- **check**: cria constraint que valida as linhas existentes. Se o dado de prod violar, a migration aborta.
- **acesso**: troca policy ou revoga grant. Não mexe em dado, mas muda quem lê e quem grava.
- **sem tx**: o arquivo não tem `begin; … commit;`. Uma falha no meio deixa estado parcial.

| # | ARQUIVO | ORIGEM | ATENÇÃO |
|---|---|---|---|
| 1 | `20260424121550_initial.sql` | ex-001, schema inicial | sem tx; insere a empresa "SUA EMPRESA AQUI LTDA" |
| 2 | `20260424121551_rls_policies.sql` | ex-002 | sem tx; acesso |
| 3 | `20260424121552_storage_buckets.sql` | ex-003 | sem tx; acesso (apaga e recria policies do Storage) |

| # | ARQUIVO | ORIGEM | ATENÇÃO |
|---|---|---|---|
| 4 | `20260511151924_revisao_schema.sql` | ex-004, Sprint 3 | **DESTRUTIVA**: apaga 13 colunas de `obras` e 6 de `orcamentos` depois de copiar para `clientes`, sem `if exists`; dado; check |
| 5 | `20260512135125_rls_clientes.sql` | ex-005 | sem tx; acesso |
| 6 | `20260512141810_fix_calcular_valores_obra.sql` | ex-006 | sem tx |
| 7 | `20260512151918_progresso_itens_obra.sql` | ex-007 | sem tx; tem de vir depois da 4 |
| 8 | `20260901120909_documentos_processamento.sql` | ex-008, automação | |
| 9 | `20260901120910_contatos_whatsapp.sql` | ex-009, automação | |
| 10 | `20260901120911_storage_bucket_documentos_processamento.sql` | ex-010, automação | sem tx; acesso |
| 11 | `20260905180000_propostas_historico.sql` | 4.6 | check |
| 12 | `20260905190000_fix_calcular_valores_obra_ambiguidade.sql` | 4.1 / 4.3 | |
| 13 | `20260908120000_orcamentos_historico.sql` | 4.6 | check |
| 14 | `20260921150000_contatos_canal.sql` | automação, fase 1 | dado (preenche `canal` dos documentos existentes); check |
| 15 | `20260922120000_recalculo_valor_por_itens.sql` | 5.6 | |
| 16 | `20260922130000_recalculo_valor_trava_pai.sql` | 5.6 | |
| 17 | `20260922140000_recalculo_valor_security_definer.sql` | 5.6 | acesso |
| 18 | `20260923120000_itens_reordenar_ajustar.sql` | 5.7 | acesso |
| 19 | `20260923130000_auditoria_eventos.sql` | auditoria (ver 13.2) | acesso |

| # | ARQUIVO | ORIGEM | ATENÇÃO |
|---|---|---|---|
| 20 | `20260923131000_auditoria_referencia.sql` | auditoria (ver 13.2) | acesso |
| 21 | `20260923160000_contratos_desconto_historico.sql` | 6.1 / 6.2 | check; acesso |
| 22 | `20260923161000_itens_de_contrato.sql` | 6.1 | acesso |
| 23 | `20260923162000_gerar_contrato_de_proposta.sql` | 6.2 | acesso |
| 24 | `20260924100000_contatos_nome.sql` | automação, fase 7 | sem tx |
| 25 | `20260924110000_execucao_varias_por_item.sql` | 7.4 | |
| 26 | `20260924120000_execucao_nomes_checks.sql` | 7.1 / 7.6 | renomeia CHECKs pela definição |
| 27 | `20260924130000_execucao_data_fim_total.sql` | 7.4 | |
| 28 | `20260925100000_documentos_revisao.sql` | automação, fase 7 | check |
| 29 | `20260925120000_execucao_medicoes.sql` | 8.4 | dado (cria o histórico das medições existentes) |
| 30 | `20260925130000_documentos_conferencia.sql` | automação | sem tx |
| 31 | `20260928100000_recalculo_valor_sem_deadlock.sql` | 8.5 | |
| 32 | `20260928110000_automacao_eventos.sql` | automação (log) | |
| 33 | `20260928120000_automacao_eventos_ordem.sql` | automação (log) | sem tx |
| 34 | `20260929100000_nf_cancelada_terminal.sql` | 9.4 / 9.6 | NF cancelada passa a ser imutável |
| 35 | `20260930200000_pagamento_parcela_cancelada_e_comprovante.sql` | 10.6 | acesso |
| 36 | `20260930210000_criar_acordo_com_parcelas.sql` | 11.2 | acesso |

| # | ARQUIVO | ORIGEM | ATENÇÃO |
|---|---|---|---|
| 37 | `20260930220000_converter_acordo_em_nf.sql` | 11.4 | acesso |
| 38 | `20261005120000_usuarios_protecao_perfil.sql` | 13.1 | sem tx; acesso; **corrige falha de segurança que existe em gc-prod** |
| 39 | `20261006100000_auditoria_historico_nf.sql` | 13.2 | acesso |
| 40 | `20261007100000_rls_revisao_13_3.sql` | 13.3 | acesso; **corrige falha de segurança que existe em gc-prod** |
| 41 | `20261007110000_rls_comprovante_upload.sql` | 13.3 | acesso; **corrige falha de segurança que existe em gc-prod** |
| 42 | `20261007120000_recalculo_valor_definer_de_volta.sql` | 13.3 | acesso |
| 43 | `20261008100000_documentos_opcoes_obra.sql` | automação (decisão 31) | sem tx |
| 44 | `20261012100000_propostas_aditivo_de_contrato.sql` | automação (aditivo) | sem tx |
| 45 | `20261012110000_propostas_aditivo_fk.sql` | automação (aditivo) | sem tx; check; **só com o código que nomeia a FK publicado** |
| 46 | `20261019100000_pedidos_acesso.sql` | "Esqueci minha senha" (Sprint 14) | acesso |

A classificação saiu da leitura de cada arquivo ( `drop` , `alter` , `update` / `insert` fora de função, `add constraint` , `revoke` , `begin` ). As funções que fazem `update` por dentro (recálculo, cascata) não estão marcadas: só agem quando chamadas.

**Aplicar:**

```
npx supabase db push --dry-run        # lista o que vai entrar; conferir com a tabela
npx supabase db push
npx supabase migration list           # as 46 nos dois lados
```

Depois do push, rodar as contagens do passo 2 de novo: só a 4 (se entrar agora) muda dado, e `clientes` ganha as linhas copiadas de `obras` e `orcamentos` .

**A 45 e o deploy.** A FK do aditivo cria uma segunda relação entre `propostas` e `contratos` , e embed sem nome dá `PGRST201` . O código da `main` já nomeia a FK; o código que está hoje no deploy apontado para gc-prod não se sabe. Por isso a ordem é: sistema parado → migrations → deploy da `main` → fumaça.

**Fora de migration, no painel do Auth do gc-prod:**

- desligar o cadastro aberto (`disable_signup = true`), como no gc-dev no 13.3;
- conferir a validade do link de convite (em gc-dev, 1 hora).

**Tipos.** O `src/lib/supabase/types.ts` é gerado do gc-dev. Com as 46 aplicadas, o schema é o mesmo; não é preciso regenerar a partir de prod.

## 5. Variáveis de ambiente

Levantadas do código (`process.env` em `src/`) em 2026-10-26. Vão nas Project Settings da Vercel, ambiente **Production**, com os valores do gc-prod.

| VARIÁVEL | ONDE É LIDA | OBRIGATÓRIA | SEM ELA |
|---|---|---|---|
| `NEXT_PUBLIC_SUPABASE_URL` | `src/lib/supabase/*` | sim | nada abre |
| `NEXT_PUBLIC_SUPABASE_ANON_KEY` | `src/lib/supabase/*` | sim | nada abre |
| `SUPABASE_SERVICE_ROLE_KEY` | `src/lib/supabase/admin.ts` (gestão de usuários, pedidos de acesso), rotas de ingestão, documentos | sim | gestão de usuários avisa e não convida; ingestão recusa |
| `INGESTAO_TOKEN` | `src/app/api/ingestao/*` | se houver automação | as rotas do n8n devolvem 401 |
| `INGESTAO_PROFILE_ID` | `src/app/api/ingestao/{proposta,contrato}` | se houver automação | a ingestão não tem autor; é o id do usuário da automação **em gc-prod** |
| `N8N_DOCUMENTO_WEBHOOK_URL` | `src/app/(app)/documentos/actions.ts` | se houver automação | o envio pela tela registra mas não aciona a leitura |
| `N8N_DOCUMENTO_TOKEN` | idem | se houver automação | idem |
| `TELEGRAM_BOT_TOKEN` | `src/lib/telegram.ts`, documentos | se houver automação | o bot não responde |
| `TELEGRAM_ADMIN_CHAT_ID` | documentos | não | usa o padrão do código (`-5521155438`) |

`SUPABASE_SERVICE_ROLE_KEY` nunca leva o prefixo `NEXT_PUBLIC_`. O `README.md` da raiz ainda lista só três variáveis e diz que a service role "ainda não é consumida": está desatualizado.

Variáveis só dos scripts (não vão para a Vercel): `VALIDACAO_EMAIL`, `VALIDACAO_SENHA` (plano de validação e autor padrão da importação) e `NEXT_DIST_DIR` (pasta do build da validação).

**A automação em produção é uma decisão em aberto.** Hoje o deploy `gc-sistema-nine.vercel.app` (fork `brenocmlo/gc-sistema`) aponta para o **gc-dev**, e é ele que o n8n chama. Os documentos de entrega também citam um deploy do repositório principal apontando para gc-prod, com login da Vercel na frente. Antes do go-live a equipe precisa decidir qual projeto da Vercel vira produção, e se o n8n e o bot do Telegram passam a apontar para ele.

## 6. O que muda no `gc-dev-guard` para permitir prod

**Descrito, não feito.** Hoje `exigirGcDev()` aborta qualquer ref diferente de `gzbmhgnpoehormnidmgg`, e o `validar.sh` confere o `.env.local` antes das camadas que conectam. A proposta, para quando o Breno autorizar:

1. **O plano de validação continua só em gc-dev.** As camadas criam e apagam dado (`VALIDA-*`, `RUN-*`) e usam perfis de teste sem senha (`sessao-dev.mjs`). Nada disso roda em prod.
2. **Uma função nova, `exigirAmbiente(url, acao, { prod })`**, ao lado da atual, usada só pela importação e pela conciliação. Ela aceita o ref de prod quando as três condições valem:
   - a flag `--prod` na linha de comando;
   - `GC_PROD_AUTORIZADO=zondscnfosubxutpshux` no ambiente da execução, nunca no `.env.local`;
   - uma confirmação digitada (o ref) no terminal.
3. **Um `.env.prod.local` separado**, fora do `.env.local`, carregado com `--env-file`. O comentário do guard diz por quê: com as duas credenciais no mesmo arquivo, prod fica "a um ctrl+C de distância".
4. A trava some quando a janela fecha: a variável não fica salva em lugar nenhum.

## 7. Usuários reais pelo convite por link

O sistema não manda e-mail (SMTP padrão da Supabase: 2 e-mails por hora, só para a equipe; ver 13.1 e `esqueci-minha-senha-status-entrega.md`). O convite é um link que o admin copia e manda.

**O primeiro admin** não tem quem o convide. Ele entra pelo painel:

1. Authentication → Users → *Add user* (e-mail e senha temporária).
2. A empresa: a migration 1 cria "SUA EMPRESA AQUI LTDA". Corrigir nome e CNPJ (pelo SQL ou depois em `/configuracoes`).
3. O perfil: a parte 3 de `supabase/criar_usuario_admin.sql` (linha em `public.profiles` com a empresa e `perfil = 'admin'`). Não gravar senha no arquivo, que é versionado.

**Os outros:** o admin entra no sistema → Configurações → Usuários → *Convidar*, com nome, e-mail e perfil (admin, comercial, financeiro, medição, produção ou visualizador). A tela devolve o link, que leva a `/convite` e à tela de definir senha. O link usa o endereço por onde o admin acessou: por isso os convites saem **depois** do domínio próprio (passo 11), ou vão com o endereço provisório. Link vencido: *Gerar link de acesso* na mesma tela.

**Conferir:** cada pessoa entra com a própria senha e vê só o que o perfil permite. A lista de quem entrou fica na seção 15.

## 8. Importação

Script e regras: `docs/sprint-14/14.2-status-entrega.md`. Na ordem:

1. Ajustar `scripts/importacao/mapa-planilha.json` à planilha real e conferir a simulação em **gc-dev** até os totais baterem (14.2 e 14.3). Isso é pré-condição, não parte da janela.
2. Na janela, com o guard liberado (seção 6):

```
# simulação contra prod: lê, monta e relata; não grava
node --env-file=.env.prod.local scripts/importacao/importar-planilha.mjs "Gestão
Contratos.xlsx" \
  --prod --autor <e-mail do admin real> --ate <data de corte> --relatorio importacao-
prod.json

# gravação (idempotente: rodar de novo cria 0)
node --env-file=.env.prod.local scripts/importacao/importar-planilha.mjs "Gestão
Contratos.xlsx" \
  --prod --gravar --autor <e-mail do admin real> --ate <data de corte> --relatorio
importacao-prod.json
```

- `--autor` é obrigatório em prod: o padrão é `VALIDACAO_EMAIL`, o usuário da validação, que não existe lá. A empresa dos registros é a do autor.
- `--prod` ainda não existe no script (seção 6).
- As linhas ignoradas do relatório têm de ser as mesmas da última simulação em gc-dev.
- `--desfazer` existe e apaga o que a planilha importou. Em prod, ele é o plano B antes de chegar ao backup.

## 9. Conciliação

A conciliação é o bloco 14.3: os totais do sistema têm de fechar com os da planilha na data de corte. Os valores de referência do card são R$ 627.959,83 de NF emitida, R$ 489.498,13 recebidos e R$ 2.060.290,51 de FD a descontar; a equipe ainda não confirmou se valem para a data de corte (pergunta da seção 3). O roteiro e o script da conciliação são do 14.3 e entram aqui quando forem entregues. Não fechou: `--desfazer`, corrigir em gc-dev e repetir. Fechou: **congelar a planilha** (só leitura, com a data no nome do arquivo e o aviso de que o sistema é a fonte).

## 10. Fumaça

A fumaça do repositório (`bash scripts/validacao/validar.sh fumaca`, 22 passos, 13.5) **não roda em prod**: cria registros `VALIDA-FUMACA-*`, usa perfis de teste e passa pelo `gc-dev-guard`. Em prod, a fumaça é manual e não grava nada:

1. `/login` abre pelo domínio com HTTPS; senha errada é recusada.
2. O admin vê Dashboard, Obras, Propostas, Contratos, Execução, NFs, FD e Configurações.
3. Um usuário de cada perfil entra e é redirecionado do que não pode (o mapa é o de `scripts/validacao/matriz-rls.json`).
4. Uma obra importada abre com propostas, itens, execução e NFs; os totais conferem com a conciliação.
5. Um export XLSX baixa.
6. Se a automação for para prod: um documento de teste pelo bot chega a `/documentos`.

Gravar em prod para testar (criar e apagar uma NF, por exemplo) é decisão do Breno.

## 11. Domínio próprio e HTTPS na Vercel

1. Vercel → projeto de produção → Settings → Domains → *Add* o domínio escolhido.
2. No DNS do domínio: registro `A` para o apex ou `CNAME` para o subdomínio, com o valor que a Vercel mostrar na tela (não copiar de outro lugar).
3. A Vercel emite o certificado sozinha depois que o DNS propaga. Conferir:

```
curl -sI https://<dominio>/login | head -1      # HTTP/2 200
curl -sI http://<dominio>/login  | head -3      # redireciona para https
```

4. Tirar a proteção por login da Vercel do ambiente de produção (os documentos registram que o deploy de prod tinha login da Vercel na frente).
5. Se a automação for para prod, trocar o endereço chamado pelo n8n para o domínio novo.

O Auth da Supabase não precisa do domínio para o convite (o link é montado pelo próprio sistema e validado com `verifyOtp`). Mesmo assim, atualizar a *Site URL* do Auth para o domínio, para nenhum fluxo padrão apontar para `localhost`.

## 12. Restauração

O procedimento de restaurar, testado antes de precisar dele:

1. **Projeto descartável:** criar um projeto novo na Supabase (não o gc-dev, não o gc-prod).
2. **Restaurar o dump:**

```
pg_restore --no-owner --clean --if-exists -d "$RESTORE_DB_URL" completo.dump
```

Erros de objeto do sistema da Supabase (`auth`, `storage`, extensões) são esperados num restore completo. Alternativa mais limpa: `schema.sql` e depois `dados-public.sql` com `psql`. 3. **Ou pelo painel:** Database → Backups → *Restore* (sobre o próprio projeto) ou PITR, se o plano tiver. 4. **Conferir:** as contagens do passo 2 batem com as anotadas; `npx supabase migration list` mostra o mesmo histórico. 5. **Storage:** subir os arquivos baixados para os mesmos buckets e caminhos. 6. **Medir o tempo** do restore e anotar aqui: é o tempo mínimo de sistema parado num incidente. 7. Apagar o projeto descartável.

## 13. Monitoramento

Comparativo para a equipe decidir (pergunta da seção 3). Nenhum dos dois está instalado: não há `@sentry/*` no `package.json` nem `vercel.json` no repositório.

| | SENTRY | LOG DA VERCEL |
|---|---|---|
| Instalação | pacote `@sentry/nextjs`, arquivos de config e uma variável (DSN) | nada no código |
| Erro no navegador | sim, com a linha e o usuário | não |
| Erro no servidor | sim, agrupado por tipo | sim, como linha de log |
| Aviso por e-mail ou chat | sim, por regra | depende do plano da Vercel (alertas e *log drains*) |
| Retenção | conforme o plano do Sentry | curta nos planos menores |
| Custo | plano gratuito com limite de eventos | já incluso no projeto |

O sistema já tem o próprio log da automação (`automacao_eventos`, tela `/logs`, 13.2), que cobre o n8n e o bot, mas não os erros da aplicação.

## 14. Rollback

Não há migration de volta (*down*). Algumas trazem o desfazer comentado no fim do arquivo (`execucao_medicoes`, `nf_cancelada_terminal`, `pagamento_parcela_cancelada_e_comprovante`), mas a 4 não tem volta.

| O QUE DEU ERRADO | COMO VOLTAR |
|---|---|
| Deploy novo com erro | Vercel → Deployments → *Promote to Production* (ou *Instant Rollback*) do deploy anterior |

| O QUE DEU ERRADO | COMO VOLTAR |
|---|---|
| Importação errada | `importar-planilha.mjs --desfazer` com os mesmos argumentos; conferir 0 restantes |
| Migration abortou no meio | as que têm `begin/commit` desfazem sozinhas; as "sem tx" (tabela da seção 4) pedem conferência à mão e, na dúvida, o backup |
| Schema ou dado de prod perdido | restauração (seção 12) a partir do backup do passo 2 |

**Ponto sem volta:** depois que usuários reais começam a lançar no sistema, voltar ao backup apaga o que eles lançaram. Até esse momento, a planilha continua sendo a fonte, e o rollback é barato.

## 15. Registro da execução

Preencher na janela. Cada linha com data, hora e quem fez.

| PASSO | FEITO | EVIDÊNCIA |
|---|---|---|
| 1. Autorização e reativação | — | |
| 2. Backup e contagens | — | |
| 3. Repair | — | |
| 4. Migrations (46) | — | |
| 5. Variáveis na Vercel | — | |
| 6. Guard liberado na janela | — | |
| 7. Usuários | — | |
| 8. Importação | — | |
| 9. Conciliação e planilha congelada | — | |
| 10. Fumaça | — | |
| 11. Domínio e HTTPS | — | |
| 12. Teste de restauração | — | |
| 13. Monitoramento | — | |